

HW17-Homework17

T2:

CEVALLOS PAGUAY MATEO DAVID
CHALAN YAMBERLA KEVIN ANTHONY
CHICAIZA SAMBA CHIFERNANDO DANIEL
COLLAGUAZO NAZATE PABLO JOSE

Insp. Team

2 4

T4:The POOwer Rangers of Programming. **Leader:** PASTOR HERNANDEZ MATHEWS SAID

Project: Finvory

GitHub: <https://github.com/mthws-hub/T4-OPP-FINVORY->

Database: customer.csv, supplier.csv , finvorydata.json

Inspector:T2 , **Leader:** CEVALLOS PAGUAY MATEO DAVID

Rubric Evidence

	evaluation/10	observations:
1 Overview	10	Clear and concise
2 SRS	10	Complete and detailed
3. Prioritized Features	10	Well organized priorities
4. Tasks	10	Clearly defined tasks/ Graficaly
5. Plan	10	Excellent project planning
6. Architecture	10	Logical and structured
7. Use Case	10	Accurate and clear
8. Class Diagram	10	Correct and coherent
9. Mockups (Prototype: CMD,)	10	Creative and consistent
10. functionality validation	9	A validation was missing in the ID; the image is attached

1 Overview

Overview

In the field of inventory management, companies need to keep a detailed record of all product movements: when they are sold, when they are returned, and at what price. Additionally, the system will maintain a complete record of customers, allowing tracking of their purchases and interaction history. An efficient inventory system helps reduce losses, optimize costs, and improve customer service through comprehensive management of products, suppliers, and customers. For this reason, our client Manolo Rivadeneira, owner of the company "FRICO", which specializes in the marketing of brake pads for vehicles, requested us to create a system that can cover all the points mentioned above.

The overview section provides a clear and concise description of the project objectives. It effectively summarizes the purpose and scope of Finvory. The content is well-structured and easy to understand. Excellent presentation and clarity throughout. 10/10

2 SRS

The system shall:

- Manage inventories of returned items (defective or due to an incorrect purchase) in a temporary obsolete inventory until their final destination is determined: reassignment to a new inventory with a new price or disposal.
- Assign a location for the main inventories managed by the system, so that the customer can find the product they need.
- Allow the Company Account to assign three different prices to each product based on the type of buyer according to the benefits each one has. In addition, these pricing rules can be modified by the Company Account whenever required.
- Allow the company's client to graphically explore the inventory and, if desired, contact the company via the phone number that will be displayed.
- Allow the customer, when generating an invoice, to search for the client using the parameter they need (name, RUC, business name) so that the corresponding data is filled in automatically, and the seller only has to enter the products to be sold and the total is calculated automatically.
- The system must automatically calculate the gross revenue generated by the company account. The TotalGrossProfile will display the cumulative total of all recorded sales, while the TotalGrossDay will reflect the gross revenue corresponding only to the current day.
- Enable the selection of the payment method, whether cash, transfer, or check, to provide the company's customer with greater

The SRS document is complete and detailed, covering all functional and non-functional requirements.

Each requirement is well-defined and logically organized.

The use of technical language is precise and appropriate.

It demonstrates a strong understanding of Finvory's system design. 10/10

3. Prioritized Features

Prioritized Features

High

- **The system must automatically calculate the gross revenue generated by the company account.**
The TotalGrossProfile will display the cumulative total of all recorded sales, while the TotalGrossDay will reflect the gross revenue corresponding only to the current day.
- **Allow the customer, when generating an invoice, to search for the client using the parameter they need (name, RUC, business name)**
So that the corresponding data is filled in automatically, and the seller only has to enter the products to be sold and the total is calculated automatically.
- **Allow the Company Account to configure multiple tax rates (egg., VAT, service tax) in accordance with current legal regulations.**
The system should automatically apply these rates to invoices and update all related calculations when tax parameters are modified.
- **Manage inventories of returned items (defective or due to an incorrect purchase)**
in a temporary obsolete inventory until their final destination is determined: reassignment to a new inventory with a new price or disposal.

Medium

- **Allow the Company Account to assign three different prices to each product**
based on the type of buyer according to the benefits each one has. In addition, these pricing rules can be modified by the Company Account whenever required.
- **Generate and display three types of reports-sales, customers, and suppliers using line charts.** The sales report identifies the most popular products and peak demand periods; the customer report analyzes purchase quantity and frequency to identify the most active customers; and the supplier report evaluates performance and supply frequency. All reports can be exported to Excel for further analysis and data backup.

The prioritized features are clearly identified and well justified.

The team effectively distinguishes between essential and optional functionalities.

This prioritization supports an efficient development process.

Excellent organization and attention to detail are evident. 10/10

5. Plan

Project Plan

Team: 4 Programmers (Mathews Pastor, Joseph Medina, Arcelys Otavalo, Maryuri Quiña)
Total Duration: 5 weeks (Oct 10 - Nov 11)

Week 1: (Oct 10 - Oct 16) - Sprint 0: Planning and Design

- Task (All): Requirements Analysis (Study Prioritized Features.docx).
- Task (All): Architecture Design (Decision on MVC, Hybrid JSON+CSV persistence, data/ and utils/ folders).
- Task (All): UML Class Diagram Creation and API definition for the classes.

Week 2: (Oct 17 - Oct 23) - Sprint 1: Structure and Persistence

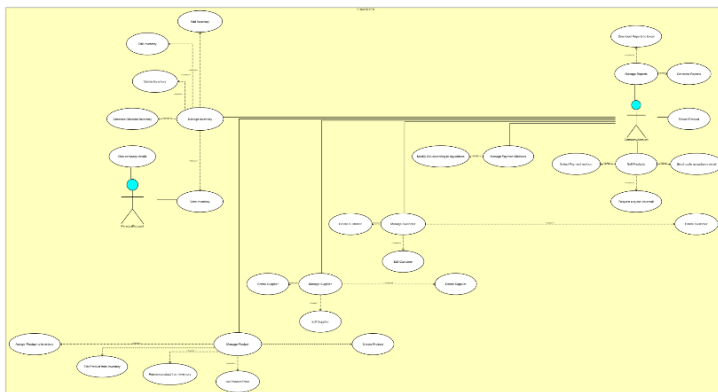
Mathews Pastor (Tech Lead): Commit 1: Program the FinvoryApp, FinvoryController, FinvoryView skeleton.

- Joseph Medina: Program all POJO Models (the 15 classes: Address, Customer, Product, etc., attributes and constructors only).
- Arcelys Otavalo: Program Database.java (the load/save JSON logic) and FinvoryData.java.

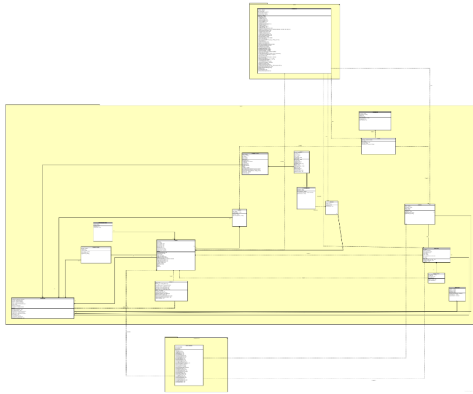
*The project plan is comprehensive and well-timed.
All milestones are clearly established and achievable.
It shows a logical sequence of development stages.
Excellent time management and coordination are evident. 10/10*

6. Architecture

*The architecture is well-designed and logically structured.
It reflects a solid understanding of software components.
Connections between modules are clear and efficient.
The diagram and explanation show high technical accuracy. 10/10*

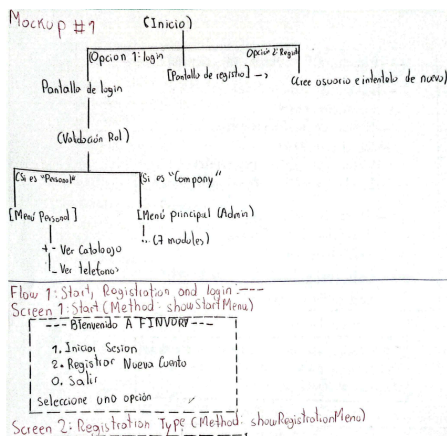


*The use cases are complete, clear, and easy to interpret.
Each interaction between user and system is well defined.
They effectively represent **Finvory**'s functional behavior.
The documentation shows strong analytical thinking. 10/10*



The class diagram is accurate and logically consistent. Relationships between classes are clearly represented. It demonstrates an excellent grasp of object-oriented design. The layout is clean, structured, and professional. 10/10

9. Mockups (Prototype: CMD,)



The mockups are creative, visually appealing, and consistent. They accurately represent the intended interface and features. Design elements show coherence with **Finvory**'s concept. Excellent attention to user experience and detail. 10/10

Observation

10. functionality validation

Opcion: 1

INFO: Cliente 'kevin' creado con exito!

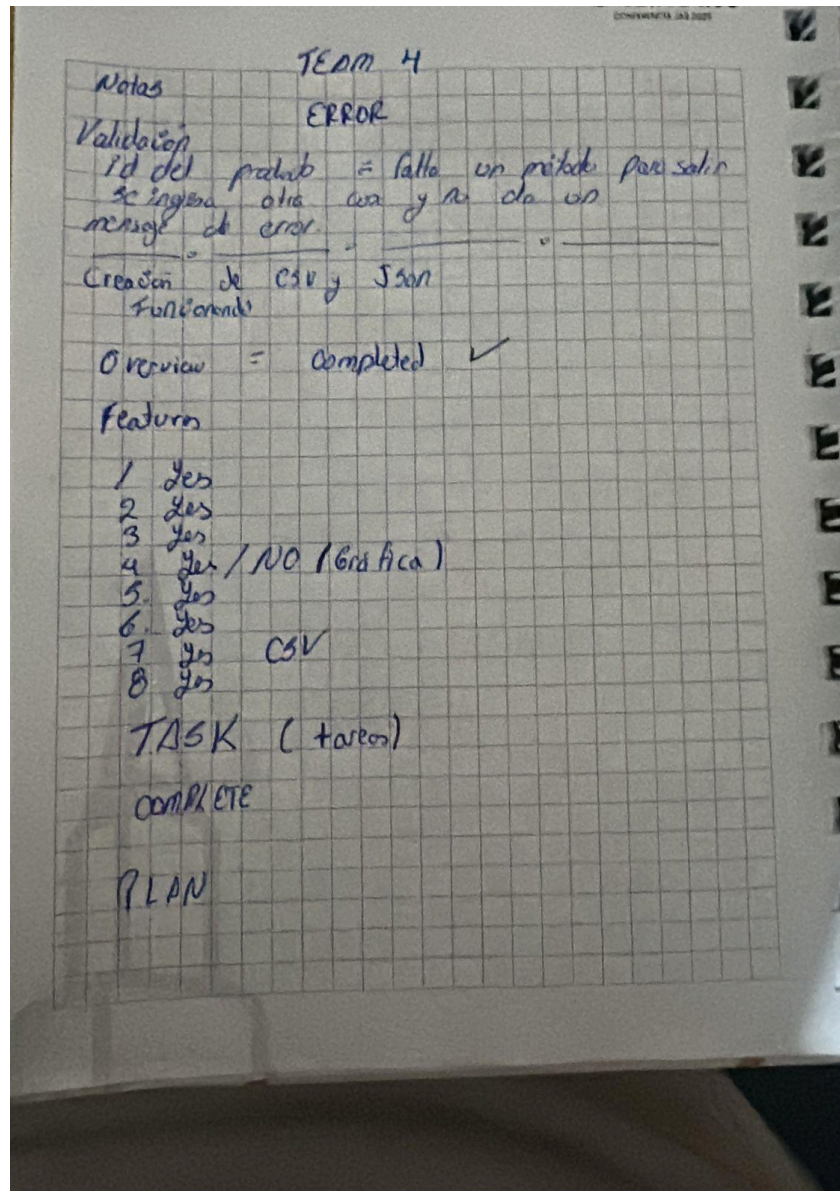
INFO: Cliente encontrado: kevin

-> Ingrese ID del producto (o 'fin' para terminar): 1

-> Ingrese ID del producto (o 'fin' para terminar): 1

Note.- In the displayed output, it can be observed that a validation for the product ID was missing. The system allows the same ID to be entered repeatedly without checking its validity or preventing duplication. Implementing an ID validation would ensure data consistency and prevent redundant entries. This issue has been identified and the corresponding image has been attached as evidence. The validation should be added to improve reliability in user input handling.

Notes



PLAN
PRIORITIZED

DATES COMPLETED
DESIGN UML

USE CASE → OK

CLASS DIAGRAM

CLASS OK ✓

ATTRIBUTES OK ✓

METHODS OK ✓

UNIT TEST - NOTHING

MOCKUP = PAPER OK ✓

CMD OK

VALIDATION OK

10	GRADE
1	10
2	10
3	10
4	10
5	10
6	10
7	10
8	10
9	10
10	9